

Summer Training Program WEEK 2 - DAY 7

Topics discussed

- Recursion
- Representation of recursion in memory
- Search, sort and filter operation

Recursion

It is that situation when a function calls itself.

We have to keep in mind the starting and the ending condition of the recursive logic.

Eg:

To find the maximum in a list using recursion:

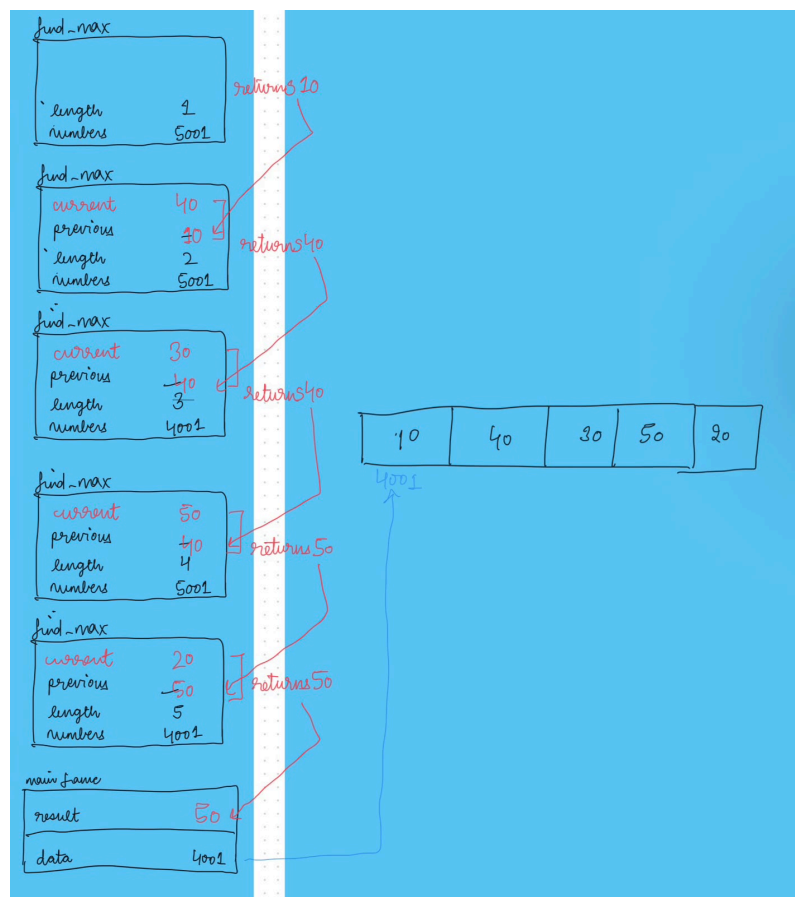
```
1 data = [10,40,30,50,20]
2
3 def find_max(numbers, length):
4     if length == 1:
5         return numbers[0]
6     else:
7         previous = find_max(numbers, length - 1)
8         current = numbers[length - 1]
9
10        if current > previous:
11            return current
12        else:
13            return previous
14
15 result = (find_max(data, len(data)))
16 print("The maximum value in the list is:", result)
17
```

Here the function find_max() calls itself again in the line 10.

OUTPUT:

The maximum value in the list is: 50

Memory representation:



Another program:

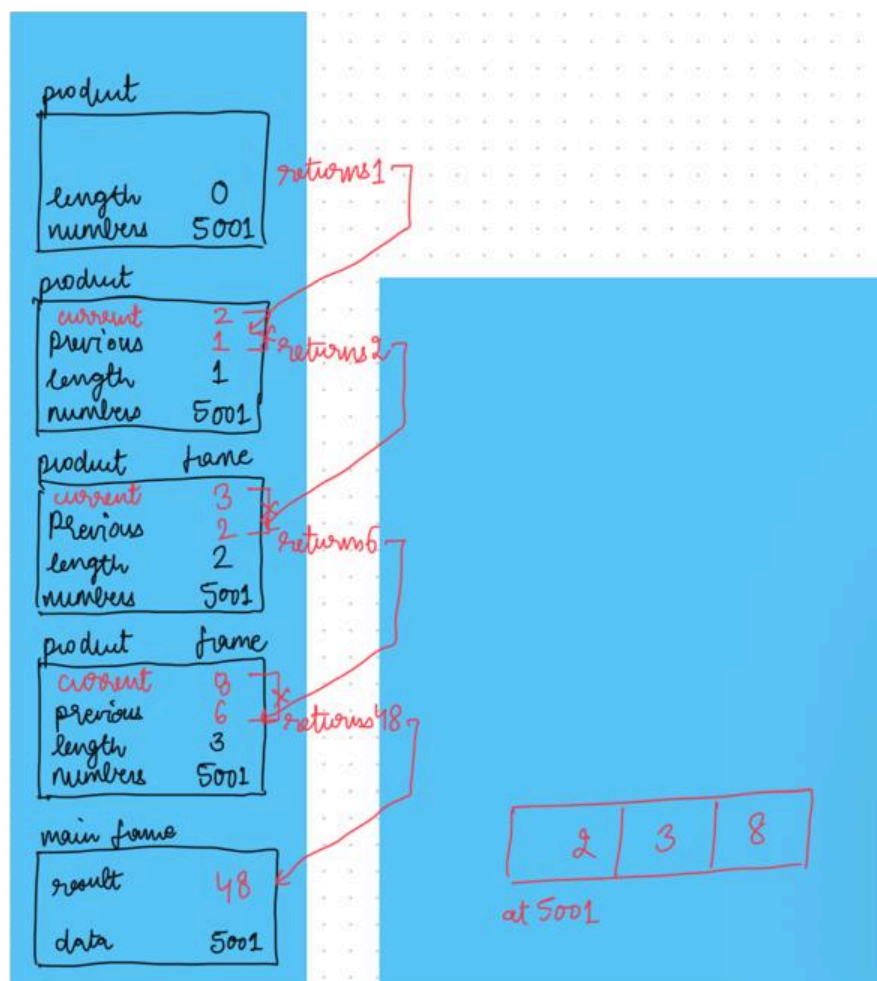
To find the product of numbers in a list:

```
1 def product(numbers, length):
2     if length == 1:
3         return 1
4     else :
5         previous = product(numbers, length - 1)
6         current = numbers[length - 1]
7         return current * previous
8
9
10 data= [2,3,8]
11 result= (product(data, len(data)))
12 print("The product of the values in the list is:", result)
```

OUTPUT:

The product of the values in the list is: 24

Memory representation:



Search operation:

Simple linear search logic:

We compare one element with the element to be searched, and then the next one with element to be searched and keep iterating it until the end of the list

```
1 def search(numbers, number_to_search):
2     for number in numbers:
3         print('[LOG] comparing:', number, 'with', number_to_search)
4         if number == number_to_search:
5             print('number found:', number)
6             break
7     else:
8         print('Number Not Found')
```

We can also implement linear search using stars and kwargs

```
11 # linear search using star and quargs
12
13 def search (*numbers, **number_to_search):
14     for number in numbers:
15         if number == number_to_search['key']:
16             print("The number", number_to_search['key'], "is found in the list.")
17             break
18     else:
19         print("The number", number_to_search['key'], "is not found in the list.")
20
21
22 search(10, 20, 30, 40, 50, key=int(input("Enter the number to search: ")))
```

OUTPUT:

```
Enter the number to search: 30
The number 30 is found in the list.
```

Sort operation:

It is used to sort elements in increasing or decreasing order.

Simple bubble sort algorithm:

```
3 def sort(data):
4     for i in range (len(data)):
5         for j in range (len(data)-1-i):
6             if data[j] > data[j+1]:
7                 data[j], data[j+1] = data[j+1], data[j]
8
9
10 numbers=[10, 30,20,5,15]
11 result= sort(numbers)
12 print("The sorted list is:", numbers)
```

OUTPUT:

```
The sorted list is: [5, 10, 15, 20, 30]
```

Filter operation:

It is a special use case of search operation.

We search multiple elements within a restraint.

Simple filter logic:

```
8 numbers = [10, 30, 20, 5, 15]
9
10 for number in numbers:
11     # Filtering
12     if number > 15:
13         print(number)
```

ASSIGNMENT: code the search, sort and filter operations for a list of flights

GURASSIS KAUR

g.assiskaur@gmail.com